CS6370: Natural Language Processing
Assignment #1

NAME: Swathi Shree Narashiman
ROLL NO: EE22B149

MARKS: 25
DUE: Mar 12, 23:59

Problem 1 — 7 marks

Part 1: Sentence Segmentation

1. Suggest a simplistic top-down approach to sentence segmentation for English texts. Do you foresee issues with your proposed approach in specific situations? Provide supporting examples and possible strategies that can be adopted to handle these issues.

Solution: A sentence in English usually ends with a terminal punctuation mark such as "." , '!', "?" . A simplistic top-down approach is to process text character by character and split the text into sentences whenever we hit any of these punctuation marks.
Pseudocode for the same is given below:

```
def sentence_segment(text):
    sentence = ''
    sentences = []

    for i in text:
        if i != '.' and i != '?' and i != '!':
            sentence += i
        else:
            sentence += i
            sentences.append(sentence)
            sentence = ''

    return sentences
```

The naive algorithm mentioned above will face many pitfalls some of them are below:

Sample Text: Dr. Arun's shift begins at 5 p.m. today.

The algorithm splits them into

Sentence 1: Dr.
Sentence 2: Arun's shift begins at 5 p.
Sentence 3: m.
Sentence 4: today.

To handle this we can maintain a seperate dictionary of abbreivations like { Dr., p.m., Mr. Mrs., ...}
Another pitfall is floating point digits, to handle them we can check if the period (.) is between two digits.

NLTK is one of the most commonly used packages for Natural Language Processing. What does the Punkt Sentence Segmenter in NLTK do differently from a simple top-down (rule-based) sentence segmentation approach? Additionally, compare both of the above approaches with spaCy's sentence segmentation method.

Solution:

In a top-down approach (such as the pseudocode algorithm with workarounds for pitfalls given above) has a set of rules to perform sentence segmentation. Whereas, NLTK's punkt algorithm is data-driven. That is, instead of fixed rules, it learns them implicitly from the training text. Therefore, punkt can clearly find if a period (.) is a sentence boundary, or an abbreviation (Dr.), or a floating point (3.14), or a date (3.6.2005) etc. Therefore, NLTK's approach is statistical and unsupervised.

On the other hand, spaCy's segmentation method is supervised and data driven, but has higher context awareness than the NLTK approach. However, due to the complexity of machine learning models, spaCy approach is significantly sloower.

2. Design an adversarial test suite for sentence segmentation:
   (a) Create 15 sentences that are likely to break a naive segmentation system.
   (b) Categorize them by error type (e.g., abbreviations, decimals, ellipsis, quotes, multiple punctuation, etc.).
   (c) Evaluate how many errors each segmenter makes on your test suite.

Solution: I have attached the code in the following colab notebook: Codefile

The results for the test suite I created are as follows:
Naive Segmenter Errors: 8 / 15
NLTK Punkt Errors: 1 / 15
spaCy Errors: 1 / 15

Problem 2 — 7 marks

Part 2: Tokenization

1. Suggest a simplistic top-down approach for tokenization in English text. Identify specific situations where your proposed approach may fail to produce expected results. [1 mark]

Solution: A simplistic top-down approach would be to split each sentence into tokens wherever white spaces and punctuation marks occur. But this approach would fail for example,
Don't - split as {Don, 't}
well-known - split as {well, -known}
3.14 - split as {3, .14}

2. Study about NLTK's Penn Treebank tokenizer and spaCy's tokenizer. What type of knowledge does each of them use - Top-down, Bottom-up, or a hybrid approach? Clearly justify your answer for both tokenizers. [2 marks]

Solution:

The Penn Treebank tokenizer is primarily rule-based, but the rules are derived from conventions used in the Penn Treebank corpus.Therefore it is best described as a hybrid approach.

Top-down component

- predefined regex rules
- rules for punctuation separation
- rules for quotes and parentheses

Bottom-up influence

- tokenization conventions were learned from an annotated corpus
- rules reflect linguistic patterns observed in real data

spaCy's tokenizer is also a hybrid system, but more sophisticated.

It uses:

Prefix rules punctuation before words

Suffix rules punctuation after words

Infix rules punctuation within tokens (hyphens, apostrophes)

Exception dictionaries

3. Perform word tokenization of the sentence-segmented documents using: [4 marks]
   (a) The proposed top-down method
   (b) Penn Treebank Tokenizer
   (c) spaCy Tokenizer

   State a possible scenario along with an example where:
   (a) Your approach performs better than the Penn Treebank and spaCy tokenizers
   (b) Your approach performs worse than the Penn Treebank and spaCy tokenizers

Solution:

Implementation in the code files.

(a) Scenario Where the Proposed Top-Down Tokenizer Performs Better

The proposed top-down tokenizer may perform better when the text is clean, simple, and does not contain complex punctuation or linguistic structures such as contractions or abbreviations. In such cases, simple whitespace and punctuation-based splitting is sufficient and avoids unnecessary token splitting.

Example

Sentence:

> I don't want this token to split.

Top-down tokenizer output:

`I | don't | want | this | token | to | split | .`

Penn Treebank / spaCy output:

`I | do | n't | want | this | token | to | split | .`

Explanation

In some applications such as keyword extraction, information retrieval, or simple text analysis, it may be preferable to treat *don't* as a single token rather than splitting it into *do* and *n't*. The proposed tokenizer preserves the original word form and may therefore be more suitable for such tasks.

(b) Scenario Where the Proposed Top-Down Tokenizer Performs Worse

The proposed tokenizer performs worse when the text contains linguistic structures such as contractions, abbreviations, decimal numbers, or special tokens like URLs and email addresses. Advanced tokenizers are designed to handle these patterns more accurately.

Example

Sentence:

> I can't believe it's already 3.14!

Top-down tokenizer output:

```
I | can't | believe | it's | already | 3 | . | 14 | !
```

Penn Treebank / spaCy output:

```
I | ca | n't | believe | it | 's | already | 3.14 | !
```

Explanation

The top-down tokenizer incorrectly splits the decimal number *3.14* into separate tokens and does not follow standard linguistic conventions for contractions. In contrast, the Penn Treebank and spaCy tokenizers use linguistic rules and exception handling, allowing them to correctly tokenize contractions and numeric expressions.

Problem 3 — 4 marks

Part 3: Stemming and Lemmatization

1. What is the difference between stemming and lemmatization? Clearly explain the conceptual differences between the two approaches in terms of linguistic knowledge, output form, and computational complexity. Give suitable examples to illustrate your answer. [1 mark]

Solution:

Stemming is a process that removes prefixes or suffixes from words to obtain a root word. The resulting stem may not always be a valid dictionary word. Stemming relies on simple rule-based transformations and does not use linguistic knowledge. Stemming is faster than lemmatization.

Lemmatization, on the other hand, reduces words to their *lemma* or root word, which is the base or dictionary form of the word. Lemmatization uses linguistic knowledge such as vocabulary dictionaries and morphological analysis to ensure that the resulting word is a valid dictionary form. This process is slower than stemming.

Examples

- *running* → **run** (lemmatization), **run** (stemming)
- *studies* → **study** (lemmatization), **studi** (stemming)
- *better* → **good** (lemmatization), **better** (stemming)

2. Using the word-tokenized text from the Cranfield Dataset: [3 marks]
   (a) Perform stemming using Porter's Stemmer.
   (b) Perform lemmatization using WordNet Lemmatizer.

   Compare the outputs of the two methods by analyzing:
   - Changes in vocabulary size
   - Examples of over-stemming (if any)
   - Cases where lemmatization preserves semantic meaning better

Solution: Implementation in the code files. **Vocabulary Size Comparison**

The vocabulary size after stemming is smaller because the Porter Stemmer aggressively truncates words to their root forms. Lemmatization preserves valid dictionary forms and therefore results in a larger vocabulary.

Examples of Over-Stemming

- studies → **studi** (stem) vs study (lemma)
- analysis → **analysi** (stem) vs analysis (lemma)
- university → **univers** (stem) vs university (lemma)
- connection → **connect** (stem) vs connection (lemma)

In these cases, stemming produces truncated forms that are not valid English words, whereas lemmatization preserves the correct base form.

Problem 4 — 4 marks

Part 4: Stopword Removal

1. Remove stopwords from the tokenized documents using a curated list, such as the stopword list provided by NLTK. [1 mark]
2. Suggest a bottom-up (data-driven) approach for constructing a stopword list that is specific to the given corpus of documents. Clearly explain the intuition behind your proposed method. [1 mark]

Solution:

Instead of relying on a predefined stopword list, we can construct stopwords directly from the corpus statistics.

(a) Compute the **term frequency (TF)** across the entire corpus.

(b) Compute the **document frequency (DF)**, i.e., the number of documents containing each word.

(c) Words that appear in a very large number of documents are likely to be non-informative.

(d) Select words whose document frequency exceeds a chosen threshold (for example, words that appear in more than 70–80% of the documents).

Intuition

Informative words usually occur only in some documents and therefore help distinguish between different topics. However, extremely frequent words appear in almost all documents and thus contribute little to distinguishing documents.

3. Implement the strategy proposed in the previous question and compare the stopwords obtained with those provided by NLTK on the Cranfield dataset. Your comparison should include: [2 marks]
   - Number of stopwords identified
   - Overlap between the two lists
   - Words that appear in one list but not the other

Solution:

Stopword Comparison

Number of Stopwords
Words only in the data-driven stopword list

- pressure, equations, method, number, mach, heat, solution
- velocity, theory, results, shock, flow, layer
- temperature, boundary, surface, effects, obtained, given

These words appear very frequently in the Cranfield corpus because it is a technical dataset related to aerodynamics and fluid mechanics.
Words only in the NLTK stopword list

- pronouns: *he, she, they, you, we, them*
- auxiliary verbs: *is, are, was, were, have, does*
- function words: *because, before, during, until*

The NLTK list contains general English function words that occur across many domains, while the data-driven list identifies high-frequency terms specific to the Cranfield dataset.

Problem 5 **3 marks**

Part 5: Retrieval

1. Given a set of queries Q and a corpus of documents D, what would be the total number of similarity computations required to estimate the similarity between every query and every document?

   Assume that TF–IDF vector representations are already available for both queries and documents over a vocabulary V. Clearly express your answer in terms of $|Q|$, $|D|$, and $|V|$. [1 mark]

   Solution: Let Q be the set of queries and D be the set of documents.

   For each query, we compute its similarity with every document in the corpus. Therefore, the total number of query-document pairs is

   $$|Q| \times |D|$$

   Assuming that the similarity is computed using cosine similarity between TF–IDF vectors, each similarity computation involves a dot product between two vectors of dimensionality $|V|$.

   Thus, the computational cost of one similarity computation is

   $$O(|V|)$$

   Therefore, the total computational complexity for computing similarities between all queries and documents is

   $$O(|Q| \cdot |D| \cdot |V|).$$

2. Suggest how the concept of an *inverted index* can be used to reduce the time complexity derived in the previous question. You may introduce additional variables (such as average document frequency or sparsity factors) to explain your reasoning. Provide a clear comparison between the naive approach and the inverted-index-based approach in terms of computational complexity. [2 marks]

   Solution:

   The naive approach computes the similarity between each query and every document by iterating through all $|V|$ terms in the vocabulary. However, TF–IDF vectors are typically very sparse, meaning that most entries are zero.

   An **inverted index** stores, for each term $t \in V$, a list of documents that contain that term along with their TF–IDF weights. This allows us to consider only documents that share at least one term with the query.

   Let:

   - k = average number of terms in a query
   - $df(t)$ = number of documents containing term t
   - $\overline{df}$ = average document frequency across query terms

   Using an inverted index, for each query term we only examine the documents that contain that term. Therefore, the number of documents examined per query becomes approximately

$$k \cdot \overline{df}$$

Hence, the computational complexity becomes approximately

$$O(|Q| \cdot k \cdot \overline{df}).$$

Comparison

Approach	Computational Complexity
Naive retrieval	$O(\lvert Q\rvert \cdot \lvert D\rvert \cdot \lvert V\rvert)$
Inverted index retrieval	$O(\lvert Q\rvert \cdot k \cdot \overline{df})$

Since $k \ll |V|$ and $\overline{df} \ll |D|$ in most real-world corpora, the inverted index approach significantly reduces the number of computations required for retrieval.